

A SOURCE-GROUNDED EXPLAINER

효율적인 에이전트 하네스의 구조

에이전트 하네스의 비용은 모델에 보내는 토큰 수와 그 토큰에 적용되는 단가, 사용자가 응답을 기다리는 시간으로 결정된다. 이 보고서는 한 에이전트 하네스가 이 세 변수를 어떻게 다루는지 코드에 근거해 분석한다. 요청마다 보내는 정보를 줄이는 축소 정책, 보낸 토큰을 캐시 단가로 과금받게 하는 프롬프트 캐시 규율, 요약 대기를 작업 시간과 겹치는 비동기 압축이 그 대상이다. 각 장치가 실제로 강제하는 조건과 코드만으로는 확인할 수 없는 효과를 구분해 서술한다.

코드 기반 분석

대화형 임계값 시각화

캐시 적중 산식

성능 실측 아님

세션 기록과 모델 입력의 관계

ARCHIVE → PROJECTION / SUMMARY

01 / PERSISTED

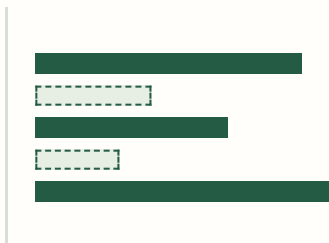
저장된 세션 원본



대화와 도구 출력이 기록되는 층이다. 요청을 줄이는 변환은 이 원본을 축소한 내용으로 덮어쓰지 않는다.

02 / PER REQUEST

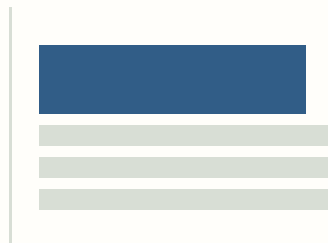
이번 요청의 투영



현재 호출에 보낼 메시지를 발체와 라벨로 구성한 결과이다. 생략된 본문은 이번 모델 입력에 포함되지 않는다.

03 / AFTER COMPACTION

요약 이후 활성 문맥



요약을 적용한 뒤에는 요약과 보존 메시지가 이후 대화의 바탕이 된다. 원본 기록과 활성 문맥은 구별된다.

그림은 동일한 세션이 저장 기록, 요청 입력, 요약 이후 문맥으로 표현되는 방식을 비교한 개념도이다. 막대의 길이는 실제 토큰량이나 압축률을 나타내지 않는다.

70%

200k 창에서 이 하네스의 기본 요약 임계값

91.808%

같은 창에서 pi 기본 기준선 · reserve 16,384

01 THE MENTAL MODEL

분석 대상과 컨텍스트의 구분

이 하네스의 컨텍스트 관리를 설명하려면 ‘압축’이라는 말이 가리키는 작업부터 구분해야 한다. 한 작업은 저장 기록에서 이번 요청에 필요한 표현을 만드는 요청 투영이다. 다른 작업은 오래된 대화를 요약으로 대체해 활성 문맥을 재구성하는 compaction이다. 두 작업은 줄이는 대상뿐 아니라 실행 시점과 결과가 유지되는 범위도 다르다. [01]

이 구분은 하네스 효율의 세 축과 대응한다. 요청 투영과 compaction은 모델에 보내는 토큰의 양을 결정한다. 요청 사이에 앞부분을 얼마나 그대로 유지하느냐는 그 토큰에 적용되는 단가를 결정한다. 07절의 프롬프트 캐시가 이 축을 다룬다. 요약을 언제 만들고 언제 적용하느냐는 사용자가 기다리는 시간을 결정하며 05절의 주제이다. 세 축은 독립적이지 않다. 요청을 줄이는 변환이 앞부분을 바꾸면 단가가 오른다. 요약을 미리 만들어도 적용하지 못하면 대기 시간은 그대로이면서 생성 비용만 남는다.

REQUEST PROJECTION

모델 호출 단위의 정보 축소

`context` 혹은 세션 메시지를 복제한 뒤 그 복사본에 변환을 적용한다. 이 과정에서 큰 도구 출력의 중간 부분을 생략하거나, 오래된 답변의 앞부분만 남기거나, 본문을 작업 라벨로 바꾼다. 변환 결과는 해당 provider 요청의 입력이며 저장된 원본 메시지를 수정하는 결과물이 아니다.

APPLIED COMPACTION

요약에 의한 활성 문맥 재구성

Compaction은 오래된 대화 구간을 요약하고 그 결과를 보존할 최근 메시지와 함께 이후 요청의 바탕으로 사용한다. 요약의 적용은 한 번의 요청에만 영향을 주는 투영과 달리, 후속 대화가 참조하는 활성 문맥의 구성을 바꾼다.

요약 생성에는 대개 모델 요청이 든다. 백그라운드에서 미리 만들었다는 이유만으로

이 작업에는 별도의 LLM 요약 호출이 필요하지 않다. 다만 문자열 처리와 토큰 추정에 드는 실행 비용은 존재한다. 정보 보존의 관점에서도 원본이 남는다는 사실과 모델이 그 내용을 볼 수 있다는 사실은 별개이다. 요청에서 빠진 내용을 다시 활용하려면 재조회 등 다른 경로가 필요할 수 있다.

그 비용이 사라지는 것은 아니다. 생성과 적용 사이에 대화 상태가 바뀌어 결과를 사용하지 못한다면, 생성 비용은 지출했지만 활성 문맥을 줄이는 효과는 얻지 못할 수 있다.

예를 들어 긴 파일 조회 결과가 세션에 저장되어 있어도, 현재 요청에서는 그 자리에 `[tool result cleared]` 만 들어갈 수 있다. 이때 파일을 조회했다는 대화 구조는 일부 남지만 모델은 해당 요청만으로 생략된 코드의 세부 내용을 확인할 수 없다. 요청량 감소를 평가할 때에는 이처럼 생략된 정보가 이후 작업에서 다시 필요해지는지도 함께 살펴야 한다.

이 보고서에서 provider는 모델 호출을 처리하는 제공자 또는 그 연결 구현을 가리킨다. Job은 백그라운드 요약 작업 하나를 뜻하며 warm 결과는 미리 생성해 두었으나 아직 적용하지 않은 요약을 말한다. 이러한 용어는 코드의 식별자와 대조할 수 있도록 원형을 유지하되, 생성 중인 작업과 완료된 결과를 구별해 사용한다.

정보 보존의 범위. 요청 축소는 세션 원본을 보존하는 손실 투영으로 해석하는 것이 적절하다. 기록의 보존은 확인되지만 모델에 전달되는 의미가 모두 보존되거나 필요한 정보를 항상 다시 찾을 수 있다는 보장까지 포함하지는 않는다.

02 EXECUTION ORDER

요청 처리 단계별 실행 순서

사용자의 요청 하나는 보통 여러 번의 모델 호출로 처리된다. 응답에 도구 호출이 포함되면 도구 실행 결과를 받은 모델이 다시 응답하는 과정이 이어질 수 있다. 이 하네스는 이 흐름의 여러 지점에 훅(hook)을 두고 새 사용자 요청을 시작할 때의 요약 판단과 각 모델 호출 직전의 요청 투영을 따로 수행한다. 이 구분은 어느 시점에 사용자 작업이 요약을 기다리게 되는지 설명하는 데 필요하다. ^[02]

새 요청 시작

before_agent_start

요약 필요성의 선행 판단

판정에는 현재 사용량뿐 아니라 대기 중인 prompt와 이미지의 추정 토큰도 반영된다. 먼저 사용량과 reserve(나중을 위해 미리 떼어 두는 여유 토큰)의 합이 창에 도달하는 하드 리밋 조건을 확인한 다음, 적응 임계값과 유예 조건을 검사한다. 그보다 앞선 구간에서는 speculative 요약의 생성 조건을 확인한다. Blocking 경로를 선택하면 새 작업은 이 단계에서 요약을 기다릴 수 있다. 다만 조건에 도달했다는 사실만으로 적용이 확정되지는 않으며 breaker 상태와 provider별 소유권 등 추가 제약을 받는다.

hard limit → threshold / grace → speculative start

각 모델 호출

context

Provider에 전달할 메시지의 구성

요약 판단이 끝나면 실제 모델 요청을 만든다. 이때 도구 출력 admission, 일반 context reduction, emergency prune을 순서대로 적용한다. 뒤 단계는 앞 단계가 바꾼 메시지를 입력으로 받으므로, 같은 원문에 세 변환이 따로따로 적용되는 구조가 아니다. 이어서 provider별 replay 처리와 tool-call/result 짝 복구가 이어진다. 짝 복구는 일부 메시지가 사라진 뒤에도 요청에 필요한 호출과 결과의 짝을 정리하는 단계이다. 생략한 본문을 원문으로 되돌리는 작업은 아니다.

admission → reduction → emergency → pair repair

응답·도구 실행

message_end /
turn_end

응답 관찰과 복구 조건의 검사

요약을 적용한 뒤에는 정해진 감시 구간에서 텍스트 없는 assistant 응답을 관찰한다. 감시기는 메시지 종류와 텍스트 존재 여부에 따라 연속 카운터를 갱신하므로, 도구 호출 횟수를 그대로 실패 횟수로 취급하지 않는다. 최근 요약의 절약량이 0인 경우에는 별도의 turn-end 복구 시도 경로도 존재한다. 여기서 사용하는 turn_end 횟수를 사용자의 요청 횟수와 동일하게 해석해서는 안 된다.

에이전트 종료

agent_end

유휴 시간의 요약 준비

에이전트 작업이 끝난 뒤에는 다음 요청이 들어오기 전까지의 시간을 이용할 수 있다. 실행 모드와 설정, 중단·재시도 여부, breaker 상태가 적격하면 요약을 새로 시작하거나 기존 warm 결과를 유지한다. 생성 작업이 완료되더라도 세션이 여전히 유휴 상태이고 실제 요약 임계값 이상이어야 곧바로 적용할 수 있다. 사용자가 먼저 새 요청을 보내면 유휴 적용의 전제가 달라진다.

실행 순서의 해석. 요청 투영이 세 단계라는 사실만으로 “세 번 줄여 본 뒤에도 부족할 때 요약한다”고 설명할 수는 없다. 새 요청은 `before_agent_start` 에서 먼저 blocking compaction을 기다릴 수 있다. 요청 축소 정책과 요약 실행 정책은 서로 영향을 주지만 하나의 직렬 축소 절차로 통합되어 있지는 않다.

03

DETERMINISTIC REDUCTION

결정적 규칙에 의한 요청 축소

아래 변환은 모델이 내용을 읽고 중요도를 판단하는 요약이 아니라, 메시지의 종류·위치·크기를 기준으로 수행하는 규칙 기반 처리이다. 설명의 범위는 이 하네스가 컨텍스트를 관리하고 관련 설정이 활성화된 기본 경로로 한정한다. SDK 소유 경로와 remote compaction 대상 모델의 예외는 뒤에서 별도로 다룬다. 각 수식의 토큰 값은 구현이 사용하는 추정량이며 문자열 길이와 UTF-8 바이트 수를 혼용해서는 안 된다.

5%

ADMISSION

개별 도구 결과의 텍스트 예산 제한

Tool-admission은 도구 결과 하나가 요청 공간을 과도하게 차지하지 않도록 텍스트 예산을 제한한다. 전체 사용량과 무관하게 매 요청 실행된다. 다만 설정으로 비활성화할 수 있으므로 실행 여부는 설정에 달려 있다. 제한 대상은 도구 결과의 텍스트이며 이미지 부분이 이 예산에 맞춰 잘라내지는 않는다. [03]

```
cap = min(50,000, max(8192, floor(contextWindow * 0.05)))
```

기본 상한은 창의 5%를 기준으로 하되 8192토큰보다 작아지거나 50,000토큰보다 커지지 않도록 계산한다. 예를 들어 200k 창에서는 10,000토큰이 된다. 이 하한과 상한 때문에 작은 창과 큰 창에서는 실제 비율이 반드시 5%가 되지는 않는다.

출력이 예산을 넘으면 앞부분에 예산의 60%, 뒷부분에 20%를 배정해 발체하고 생략 마커를 넣는다. 이는 원문 길이의 앞 60%와 뒤 20%를 보존하는 규칙이 아니다. 마커를 포함한 추정량이 여전히 초과되면 추가로 줄인다. 60%와 20%만으로 최종 요청량을 확정할 수는 없다.

50%

REDUCTION GATE

오래된 대화와 도구 출력의 선택적 생략

일반 context-reduction은 사용량이 창의 절반 이상일 때 활성화된다. 이 단계에는 세 가지 변환이 포함되며 각각 보호하는 범위와 줄이는 대상이 다르다. 최근 정보를 보존한다는 목적은 공통이지만 보호 구간은 변환마다 따로 정의된다. [04]

연속 결과 접기는 오래된 구간에서 같은 종류의 read/search/shell 작업 쌍이 이어지는 경우 결과 본문을 그룹 라벨로 바꾼다. 기본적으로 최소 2개가 모여야 하며 최근 메시지 5개는 이 단계의 대상에서 보호한다. 여러 메시지를 하나로 합쳐 의미 요약문을 작성하는 것이 아니라, 개별 결과의 상세 본문을 작업을 나타내는 짧은 표현으로 치환하는 처리이다.

옛 답변 자르기는 최근 약 3000토큰의 보호 구간 밖에 있는 텍스트 전용 assistant 답변에 적용된다. 해당 답변이 800토큰을 넘으면 남길 본문의 목표를 $800 \times 0.3 = 240$ 토큰으로 계산하고 축소 마커를 덧붙인다. 기준은 원문 길이가 아니라 설정된 800토큰이므로, 원문이 2000토큰이라고 해서 600토큰을 남기는 것은 아니다. 도구 호출 등이 섞인 메시지까지 같은 방식으로 처리한다고 일반화해서도 안 된다.

옛 도구 결과 비우기는 clearable 목록에 속한 도구 결과 중 최근 6개를 제외한 옛 텍스트를 마커로 바꾼다. 여기서 최근 6개는 clearable 도구 결과만 센 수이다. 앞선 변환에서 이미 발췌되거나 라벨로 바뀐 결과를 원문으로 되돌려 주지 않는다. 이 단계의 보호는 추가 삭제를 막는 범위에 그친다.

BEFORE · 개념 예시

2000토큰의 오래된 텍스트 답변
옛 read 결과의 긴 파일 본문

AFTER · 실제 변환 형태

약 240토큰의 앞부분 + 축소 마커
[tool result cleared]

95 /
85EMERGENCY
LATCH

프롬프트 공간 초과에 대한 긴급 축소

Emergency-prune은 앞선 변환을 거친 뒤에도 메시지가 지나치게 큰 경우에 개입한다. 진입 조건은 메시지 추정량이 프롬프트용 창의 **95%를 초과**하는 것이며 이미 켜진 상태는 **85% 이하**가 되어야 해제된다. 두 판정의 분모는 모델의 전체 context window가 아니라 출력 공간을 제외한 프롬프트용 창이다. [05]

```
promptWindow = C - min(model.maxTokens, floor(C × 0.5))
engage: estimate > floor(promptWindow × 0.95)
release: estimate ≤ floor(promptWindow × 0.85)
```

유효한 양수 `maxTokens` 가 주어지면 모델 출력 한도와 창의 절반 중 작은 값을 출력 `reserve`로 확보한다. 유효 값이 없으면 이 `reserve`를 빼지 않는다. 같은 전체 창 크기라도 모델의 출력 설정에 따라 긴급 축소의 절대 토큰 경계는 달라진다. 이는 뒤에서 설명하는 `compaction reserve`와 별개의 계산이다.

긴급 축소에서는 4096바이트를 넘는 도구 결과 텍스트 블록을 앞 800자와 뒤 400자로 잘라낸다. 길이 판정의 단위는 바이트이고 보존할 발체의 단위는 문자이다. 도구 출력 예산이 여전히 초과되면 일부 블록은 생략 마커만 남도록 줄인다. 전체 메시지 추정량이 그래도 충분히 작아지지 않으면 오래된 메시지까지 제거한다. 마지막 단계는 본문 축소를 넘어 메시지 자체가 요청에서 빠질 수 있는 처리이다.

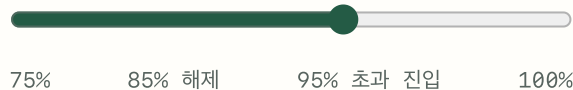
이전 상태를 반영하는 히스테리시스 판정

HYSTERESIS / STATEFUL DEMO

히스테리시스는 진입과 해제에 서로 다른 경계를 사용하는 상태 유지 방식이다. 아래 비율은 앞선 변환 이후의 메시지 추정량을 프롬프트용 창으로 나눈 값이다. 비활성 상태에서 90%에 머무르면 축소가 켜지지 않지만 96%에서 활성화한 뒤 90%로 내려오면 활성 상태가 유지된다. 85% 이하까지 내려가야 해제된다. 입력을 움직이면 이 차이를 확인할 수 있으며 실험은 `latch` 판정만 재현하고 실제 도구 출력을 삭제하지는 않는다.

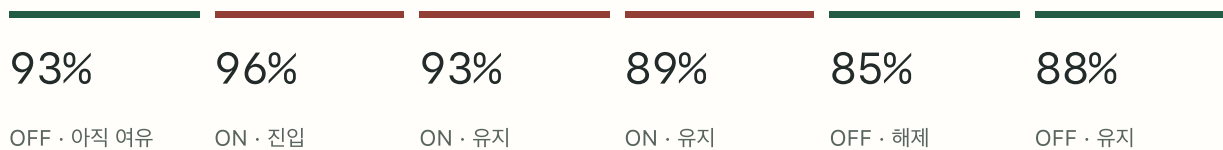
변환 후 추정 사용률

90%



OFF · 비활성

90% · 현재 비활성입니다. 95%를 초과해야 켜집니다.



진입과 해제 경계를 분리하면 사용량이 한 임계값 주변을 오갈 때 요청 형태가 반복해서 전환되는 일을 줄일 수 있다. 이는 `provider`가 재사용할 수 있는 프롬프트 앞부분을 안정적으로 유지하려는 설계상의 이유와 연결된다. 다만 실제 캐시 적중률, 다시 과금되는 범위, 총비용의 변화는 `provider`와 요청 이력에 따라 달라지므로 측정 대상으로 남는다.

적응 임계값과 토큰 예산의 계산

요청 축소와 별도로, 이 하네스는 창 크기와 직전 요약의 절약을 이용해 요약을 시작할 시점을 정하는 admission 임계값을 계산한다. 아래 실험은 기본 정책의 수식을 재현하며 실제 세션이나 모델을 호출하지 않는다. 입력 사용량에는 pending prompt의 추정량이 이미 포함되었다고 가정한다. 이 값과 도구 출력을 축소한 뒤 provider에 전달하는 요청량은 서로 다를 수 있다. [06]

차트는 여러 경계를 같은 전체 창 C 위에 배치한다. 기본 200k 예시에서는 100,000토큰부터 일반 reduction과 유틸 warm 생성의 조건이 성립하고 122,500토큰에서 pre-turn speculative 시작점에 도달한다. 요약 임계값은 140,000토큰이지만 미완성 job이 있으면 157,500토큰 미만에서 blocking을 유예할 수 있다. 따라서 생성 시작점, 적응 판단, blocking 실행의 경계를 하나의 '압축 시점'으로 묶으면 실제 동작을 놓치게 된다.

컨텍스트 예산 실험실

이 하네스 관리 경로 · compaction/speculation/grace 활성화 · breaker 정상

컨텍스트 창 C

200,000 tokens



직전 요약의 구조적 절약률

없음 / 10~50% / 10% 미만



이전 임계값을 누적 조정하지 않고 기본값에서 재계산합니다.

설정된 compaction reserve

16384

하드 리밋·grace에 사용. 작은 창에서는 값을 줄여 보세요.

모델 출력 한도 maxTokens

32000

긴급 축소의 프롬프트 공간 계산에 사용. 0은 유효 값 없음.



창 크기별 reserve scaling



미완성 speculative job 있음

판정용 컨텍스트 사용량

130,000 tokens · 65%



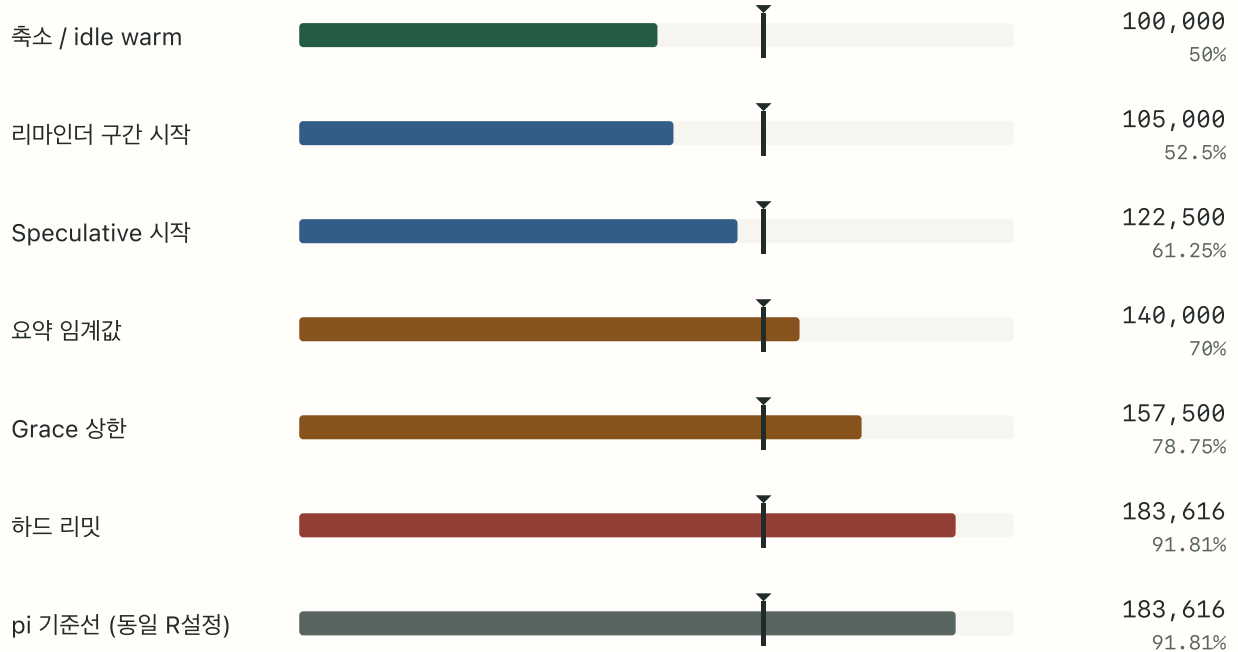
0%

50%

100%

같은 창 C를 기준으로 표시한 경계

C = 200,000



■ 요청 축소 ■ 선행 준비 ■ 요약·한계 | 입력 사용량

Speculative 시작 구간 · 기존 job 유지

이미 job이 있으면 중복 생성하지 않습니다. 임계값까지 남은 공간에서 작업을 계속할 수 있습니다. 현재 reminder 조건 구간입니다. 임계값까지 약 10,000토큰이 남았습니다. 실제 안내는 generation당 한 번입니다.

적용 임계 비율

70%

Speculative lead

17,500

유효 compaction reserve

16,384

도구 텍스트 admission cap

10,000

프롬프트용 창 P

168,000

긴급 축소 진입 / 해제

159,600 / 142,800

차트의 세로 표시는 입력 사용량과 각 경계의 상대적 위치를 나타낸다. 어떤 경계에 도달했다는 사실이 해당 작업의 즉시 실행을 보장하지는 않는다. 출력 한도를 반영한 긴급 축소 경계는 별도 수치로 제시하며 이 입력 사용량만으로 emergency-prune의 발동 여부를 판정하지 않는다. 창 밖에 놓이는 경계도 숫자는 그대로 표시하고 막대의 시각적 범위만 0~100%로 제한한다.

창 크기와 직전 절약률의 반영

기본 임계 비율은 창 크기에 따라 단계적으로 달라진다. 직전 요약이 압축 전 토큰의 50%를 넘게 절약했다면 기본값에서 5%p 낮춘다. 절약률이 그 이하이면 기본 비율을 사용한다. 매 판단은 기본 비율에서 다시 계산한다.

$C \leq 16k$	45%
--------------	-----

$C \leq 32k$	50%
--------------	-----

$C \leq 64k$	55%
--------------	-----

$C \leq 128k$	60%
---------------	-----

$C \leq 512k$	70%
---------------	-----

$C > 512k$	80%
------------	-----

Compaction reserve와 출력 reserve

Compaction reserve는 하드 리밋과 grace 등의 계산에 쓰인다. 기본 설정은 16,384토큰이다. Scaling이 활성화되면 창의 4%를 계산해 설정값과 비교하되, 창 크기로 계산하는 항은 최대 49,152토큰으로 제한한다. 설정값 자체가 그보다 크면 더 큰 설정값을 유지하므로 49,152를 모든 reserve의 절대 상한으로 읽어서는 안 된다.

```
R = max(configured,
min(floor(C × 0.04),
49,152))
```

출력 reserve는 emergency-prune에서 사용할 프롬프트 공간을 정한다. 이 계산에서는 모델의 `maxTokens` 와 창의 절반 중 작은 값을 제외한다. 두 reserve는 목적과 입력값이 다르므로, 한쪽을 변경했을 때 다른 쪽 경계도 같은 양만큼 이동한다고 가정할 수 없다.

토큰 예산 리마인더는 모델의 행동을 조정하려는 별도 장치이다. 임계값까지 남은 양이 양수이고 lead의 두 배 이내이면, 같은 generation에서 한 번 남은 예산을 안내한다. 기본 200k 예시의 구간 시작점은 105,000토큰이다. 이 안내는 탐색과 답변을 간결하게 하도록 유도하지만 이후 모델이 소비할 토큰 수를 강제하는 제한이나 실제 절약량의 보장은 아니다.

수식과 실행 상태의 구분. 계산기는 경계의 위치와 명시한 job 조건에 따른 분기를 보여 준다. 실제 실행에는 요약 결과의 유효성, 세션의 유효 여부, provider 소유권, 실패 상태가 추가로 관여한다. 이 실험의 분기 설명을 실시간 세션 관측 결과로 해석해서는 안 된다.

비동기 요약의 생성과 적용 조건

Speculative compaction은 요약을 필요로 할 가능성이 높은 시점에 먼저 생성 작업을 시작하는 정책이다. 목적은 생성 시간을 본래 작업이나 유틸 시간과 겹치게 하는 데 있다. 그러나 생성 중에도 대화가 진행될 수 있으므로, 완료한 요약이 현재 문맥에 적용 가능한지 다시 검증해야 한다. 이 때문에 생성 완료, 적용 가능, 실제 적용은 구별되는 상태이다. [07]

A / SNAPSHOT	B / GENERATE	C / VALIDATE	D / APPLY OR HOLD
기준점 확보	백그라운드 생성	현재 대화와 대조	적용 또는 보관
메시지 revision과 generation, 보존할 경계 및 branch 상태를 요약의 기준으로 잡는다.	요약 요청을 시작해 본래 작업이나 사용자 유틸 시간과 생성 시간을 겹친다.	Generation이 달라지거나 anchor가 무효하면 해당 결과를 적용하지 않는다.	유틸 적용 조건을 만족하면 적용하고 임계값 미만이면 warm 결과를 보관할 수 있다.

도식은 비동기 요약의 논리적 단계를 나타낸다. 각 구간의 가로 길이를 생성 시간, 처리 토큰 또는 비용의 비율로 해석해서는 안 된다.

선행 생성의 시작점과 유예 구간

```
T = C × effectiveRatio
L = clamp(floor(T × 0.125),
8192, 32768)
start = max(0, T - L)
G = min(T + L, C - R)
```

T는 요약 임계값이고 L은 그보다 얼마나 앞서 생성을 시작할지 정하는 lead이다. 기본 lead는 임계값의 12.5%를 내림한 뒤 8192~32768토큰 범위로 제한한다. 사용자가 lead를 설정한 경우에는 비율 계산 대신 설정값을 사용하되 같은 범위를 적용한다. 기존 job이 있으면 새 작업을 중복해서 시작하지 않는다.

유틸 생성과 유틸 적용의 분리

Idle warm-up은 에이전트 작업이 끝난 뒤 다음 요청을 기다리는 시간을 이용한다. 일반적인 warm 생성의 하한은 창 의 50%이지만 완료한 요약을 유틸 상태에서 적용하려면 **adaptive threshold 이상**이라는 별도 조건을 만족해야 한다. 200k 기본값에서 100k에 생성한 결과가 완성되더라도 사용량이 140k 미만이면 보관될 수 있다.

유틸 경로는 TUI·RPC·app-server 같은 지속 모드와 enabled 설정을 확인하고 retry/abort 여부 및 breaker 상태의 제약을 받는다. 임계값에 이미 도달한 경우의 처리가 subthreshold warm 생성보다 먼저이므로,

Grace band는 임계값에 도달한 뒤에도 미완성 job에 완료할 기회를 주는 유예 구간이다. Grace가 활성화되어 있고 입력량이 상한 G보다 작을 때 proactive blocking을 미룬다. 완료된 job에는 이 조건을 적용하지 않으며 하드 리밋 분기는 유예보다 먼저 검사한다. grace를 창의 한계까지 무조건 작업을 지속하도록 허용하는 기능으로 해석할 수는 없다.

작은 창에서는 50%보다 낮은 기본 임계값에서 작업을 시작할 수 있다. 완료 시점에도 같은 job의 결과이고 세션이 계속 유휴 상태인지 확인해야 한다.

비동기 결과의 유효성 검증에서는 generation과 revision을 구분한다. Generation은 결과가 속한 처리 세대를, revision은 메시지 상태의 변경을 구별하는 값이다. Generation이 달라진 결과는 적용하지 않는다. Revision이 달라졌더라도 요약의 기준 경계가 보존되었다면 anchor 검증을 통과할 수 있으므로, 단순히 새 메시지가 추가되었다는 이유만으로 모든 요약을 폐기하는 구조는 아니다.

Warm anchor의 검증 범위. 보존 경계가 존재하고 최신 compaction ID와 경계 앞의 entry ID 순서가 기준과 일치하면 뒤에 메시지가 추가되었어도 요약이 유효할 수 있다. 이는 본문 전체를 다시 해석하는 검증이 아니라 entry의 식별자와 경계를 확인하는 방식이다. 토큰 증가량 $> \max(\text{keepRecent}, 8192)$ 은 유휴 job을 교체할지 판단하는 조건이며 blocking 소비를 포함한 모든 적용 경로에 공통으로 적용되는 폐기 상한은 아니다.

지연 감소의 한계. 다음 요청이 요약 완료보다 먼저 들어오면 blocking 경로가 미완성 작업을 기다릴 수 있다. 생성 실패나 상태 변경으로 결과를 폐기하는 경우도 가능하다. 선행 생성은 대기를 숨길 기회를 제공하지만 모든 요청에서 체감 지연이 0이 되거나 요약 비용이 줄어든다는 보장은 아니다.

요약 이후의 단서 복원과 실패 관리

요약으로 오래된 문맥을 대체하면, 후속 작업에서 참조해야 할 파일이나 사용하던 스킴에 대한 단서도 약해질 수 있다. 이 하네스는 이를 보완하는 restoration tracker와 요약 실행의 실패를 제한하는 장치를 둔다. 다만 복원하는 정보의 내용, 실패로 집계하는 사건, 복구 시도의 허용 범위는 서로 다른 구현에 속하므로 하나의 자동 복구 기능으로 묶어 설명하기 어렵다.

복원 대상과 항목 선택 기준

Restoration tracker는 도구 호출의 입력에서 파일 경로·작업 종류·스킬 이름을 기록한다. 예를 들어 파일 항목의 내용은 `src/foo.ts (read, edit)` 와 같은 라벨이다. 파일을 다시 읽어 본문을 넣거나 스킬의 지침을 다시 로드하는 처리는 포함하지 않는다. 도구의 실행 성공을 확인한 뒤 그 결과물을 보존하는 구조도 아니므로, 라벨의 존재를 해당 작업이 성공했다는 증거로 해석해서는 안 된다. [08]

요약이 수용되면 복원할 내용을 대기 상태로 두고 다음 `before_agent_start` 에서 한 번 소비한다. 선택 순서는 최근순이 아니라 우선순위, 짧은 항목, 이름순이다. '최근에 읽은 파일을 그대로 되살린다'는 표현은 정보의 내용과 선택 기준 양쪽에서 구현과 어긋난다. 기능의 범위는 후속 작업에 관련된 대상을 상기시키는 것으로 보는 편이 정확하다.

주입 시점	압축 후 다음 <code>before_agent_start</code> 에서 한 번 소비
-------	--

기본 항목 수	파일·스킬 합계 10개
---------	--------------

항목 예산	각 5000 추정 토큰
-------	--------------

총 예산	$\min(50,000, C \text{의 } 15\%, C - \text{사용량} - \text{유효 reserve})$
------	--

선택 순서	우선순위 → 짧은 항목 → 이름순. 최근순 아님.
-------	-----------------------------

기본 예산은 파일과 스킬을 합쳐 10개이며 항목당 5000 추정 토큰을 허용한다. 총량은 50,000토큰과 창의 15%, 현재 사용량과 유효 reserve를 뺀 남은 공간 중 가장 작은 값으로 제한된다. 이 값들은 설정

RESTORED PAYLOAD · 형태 예시

```
<restored-files>
  <file label="src/foo.ts">
    src/foo.ts (read, edit)
  </file>
</restored-files>

<restored-skills>
  <skill label="programming">
    programming
  </skill>
</restored-skills>
```

선택한 항목의 내용 예산은 최종 메시지 전체 크기의 상한과 같지 않다. XML과 헤더, 별도의 checkpoint 또는 reminder가 추가될 수 있기 때문이다. 보존 메시지에 이미 있거나 이전에 복원한 라벨은 선택 대상에서 제외한다.

가능한 기본값이다. 보존 메시지에 이미 등장하거나 이전에 복원한 라벨은 제외하므로 매번 주입되는 항목은 기록 가운데 일부이다.

3 → 60 FAILURES /
SECONDS

실패 누적에 따른 실행 중단

Circuit breaker는 실패가 반복되는 동안 자동 압축을 잠시 멈추는 장치이다. 기록된 연속 실패가 3회에 도달하면 60초의 cooldown을 두고 성공이 기록되면 실패 횟수를 초기화한다. 수동 압축은 이 중단을 우회할 수 있다. [09]

여기서 기준은 '기록된 실패'이다. speculative 생성 오류는 그 자체로는 job의 오류 상태로만 남는다. 카운터에 들어가는 것은 blocking 경로가 이를 이어받아 일시적 실패로 기록한 경우이다. 실패 집계는 이벤트 소유권과 실행 경로에 따라 달라진다. Cooldown 중에는 임계값 이상의 요청에서 결정적 축소를 활용하는 경로가 있으며 이 하네스의 긴급 축소 경로도 남아 있다.

<10% OR <1024 TOKENS

비효율 요약의 집계 범위

절약량이 1024토큰 미만이거나 절약률이 10% 미만이면 ineffective로 집계한다. 그러나 현재 `shouldRejectByCap()` 은 항상 허용하는 결과를 반환한다. 비효율 카운트가 이후 압축을 차단하는 실행 제한으로 사용되지는 않는다.

수용된 요약은 이 집계에 앞서 성공으로 기록되어 breaker의 실패 횟수를 초기화한다. 따라서 비효율 카운트가 circuit breaker를 직접 작동시킨다고 설명할 수 없다. 절약률에 따른 임계값 조정, 비효율의 기록, 실패에 따른 cooldown은 연결 관계를 각각 확인해야 하는 별도 정책이다.

5 / 3 TURNS / NO-
TEXT EVENTS

요약 이후 응답 저하 감시

요약이 수용되면 5번의 `turn_end` 동안 감시 구간을 연다. 이 구간에서 텍스트 없는 assistant 메시지의 연속 카운터가 3에 도달하면 복구 압축을 시도한다. 텍스트가 있거나 감시 대상 비-assistant 메시지가 들어오면 연속 카운터를 초기화한다. [10]

복구는 한 사이클에 한 번 시도하며 여러 사이클을 통틀어 최대 3회의 제한이 있다. 시도 횟수는 복구가 수용되기 전에 증가하므로 거절된 시도도 한도를 소비할 수 있다. Breaker 등 다른 실행 제약도 받기 때문에 발동 조건을 만족했다는 사실이 복구 완료를 의미하지는 않는다.

도구 사용과 응답 저하의 구별. Assistant의 tool-only 메시지 사이에 tool result가 들어오면 연속 카운터가 초기화될 수 있다. 이 감시기는 텍스트 답변 없이 도구를 사용한 횟수 자체를 세지 않는다. 조건 평가의 입력은 assistant 응답만이 아니라 그 사이에 발생한 메시지 이벤트를 포함한다.

프롬프트 캐시 적중과 과금 단가

앞 절까지의 정책은 모델에 보내는 토큰의 양을 다룬다. 효율의 두 번째 축은 보낸 토큰 가운데 얼마가 캐시 단가로 과금되느냐이다. Provider의 prompt cache는 요청의 앞부분이 직전 요청과 동일할 때 그 구간을 재사용한다. 재사용된 토큰에는 일반 입력보다 낮은 단가가 적용된다. 하네스가 관여할 수 있는 지점은 요청을 캐시 가능한 형태로 표시하는 일, 요청 사이에 앞부분을 그대로 유지하는 일, 캐시가 만료되기 전에 다음 요청을 보내도록 시간을 관리하는 일이다. 이 절은 화면에 표시되는 적중률의 정의부터 확인한다.

[13]

FOOTER · CH

표시 수치의 정의

```
latestCacheHitRate =
  cacheRead / (input +
    cacheRead + cacheWrite)
```

화면 하단 표시줄/footer)의 CH xx.x%는 세션 누적값이 아니라 **가장 최근 assistant 메시지 하나의 비율**이다. 세션 관리자는 assistant 메시지가 기록될 때마다 이 값을 다시 계산해 덮어쓴다. 비율이 10% 미만이면 footer에 표시하지 않는다.

여기서 assistant 메시지는 사용자의 요청 단위가 아니라 provider 호출 단위이다. 한 요청 안에서 도구를 다섯 번 사용하면 assistant 메시지 다섯 개가 생기고 각각이 이 값을 갱신한다. 사용자 요청 세 번은 호출 열 번 이상일 수 있다.

SESSION INFO

누적 수치와의 차이

/session 정보 화면의 Cached (xx%)는 세션 전체의 cacheRead 합계를 전체 prompt 토큰 합계로 나눈 값이다. 이 값은 첫 호출의 완전한 miss와 압축 직후의 miss를 모두 포함하므로 footer 값보다 낮게 나온다.

두 수치는 분모의 구성이 다르므로 서로 비교해 정책의 효과를 판단할 수 없다. 특정 시점의 캐시 붕괴는 footer 값에, 세션 전체의 과금 구조는 누적값에 나타난다. 세션 정보 화면은 이와 별도로 **cache waste**를 계산해 직전 요청에 있었으나 캐시에서 읽지 못한 토큰을 집계한다.

몇 번의 호출 만에 100%에 근접하는 이유

비율이 빠르게 수렴하는 이유는 분모의 구성에 있다. 요청 하나의 prompt는 system prompt, tool 정의, 대화 이력, 새로 추가된 메시지로 구성된다. 코딩 에이전트에서 system prompt와 tool 스키마는 수만 토큰에 이르는 고정 구간이다. 매 호출에 새로 붙는 것은 사용자 메시지 수백 토큰이나 도구 결과 수천 토큰에 그친다. 직전 호출의 prompt 전체가 다음 호출의 앞부분이 되므로 분자는 직전 분모와 거의 같아진다.

호출	새로 추가된 내용	cacheRead	cacheWrite	표시 비율
1	system + tools 30,000 · 사용자 메시지 500	0	30,500	0%
2	도구 결과 3,000	30,500	3,000	91.0%
3	다음 사용자 메시지 500	33,500	500	98.5%
4	도구 결과 600	34,000	600	98.3%

표의 토큰 수는 산식을 설명하기 위한 가정값이며 실측이 아니다. 각 호출에서 `input` 은 0으로 두었다. 첫 호출은 캐시 쓰기만 발생해 0%이다. 두 번째 호출부터는 직전 prompt 전체가 읽기로 잡혀 90%대에 진입한다. 이후 분모가 커질수록 새 꼬리의 비중이 줄어 99%에 가까워진다. 이 수렴은 컨텍스트 축소 정책의 성과가 아니라 prefix cache의 산술적 성질이다.

Anthropic 경로의 체크포인트 배치

Anthropic Messages API는 명시적인 `cache_control` 표시가 있는 지점까지의 prefix를 캐시한다. 이 하네스의 `anthropic-messages.ts` 는 요청 하나에 최대 세 종류의 체크포인트를 배치한다. ^[14]

system prompt	system 블록에 <code>cache_control: {type: "ephemeral"}</code> 를 붙인다. OAuth 토큰 경로에서는 Claude Code 식별 문장을 첫 블록으로 두고 그 뒤에 실제 system prompt를 둔다.
tools	tool 정의의 마지막 항목에 표시한다. <code>supportsCacheControlOnTools</code> 가 참인 모델에 한정된다. Fireworks 경유 모델은 기본적으로 제외된다.

마지막 user 메시지	변환된 메시지 배열의 마지막 user 또는 tool-result 메시지의 마지막 블록에 표시한다. 이번 요청의 끝에 찍힌 표시는 다음 요청에서 대화 이력 전체를 캐시 prefix로 만드는 역할을 한다.
tool loop 보존	마지막 메시지가 tool-result이고 그 앞이 tool-use인 연속 구간이면 직전 user 체크포인트도 함께 유지한다. 도구 결과가 붙는 동안 앞선 캐시 경계가 사라지지 않게 하기 위한 처리이다.
effort 마커	<code>supportsMidConvoEffort</code> 모델에서는 이력 뒤에 effort 수준 마커가 붙는다. 이 마커가 체크포인트를 밀어내지 않도록 실제 마지막 user 메시지를 다시 찾아 표시한다.

캐시 보존 기간은 `cacheRetention` 설정으로 정한다. 기본은 short이며 Anthropic 문서상 5분 TTL에 해당한다. long은 `PI_CACHE_RETENTION=long` 환경 변수나 모델 설정으로 지정한다. 다만 요청 대상이 `api.anthropic.com` 이며 모델이 `supportsLongCacheRetention` 일 때에만 1시간 TTL을 요청한다. none이면 체크포인트를 붙이지 않는다. 응답의 `cache_creation_input_tokens` 는 `cacheWrite` 로, 1시간 캐시 쓰기는 `cacheWrite1h` 로 별도 기록한다.

다른 provider의 캐시 방식

체크포인트 방식은 Anthropic 계열에 한정된다. 나머지 provider는 요청에 표시를 넣는 대신 세션 식별자로 캐시 라우팅을 고정하거나, 서버가 자동으로 적용한 캐시의 사용량만 읽는다. 이 하네스는 provider마다 다른 필드를 공통 `cacheRead` 와 `cacheWrite` 로 정규화하므로, footer의 산식은 같아도 그 값이 의미하는 캐시 동작은 provider마다 다르다. ^[15]

경로	요청에 넣는 것	사용량 해석	유의점
Anthropic Messages	<code>system·tools·</code> 마지막 user 블록의 <code>cache_control</code>	<code>cache_read_input_tokens</code> , <code>cache_creation_input_tokens</code>	TTL 5분 또는 1시간. 체크포인트 위치가 곧 캐시 경계이다.
OpenAI Completions / Responses / Codex	<code>prompt_cache_key</code> 에 세션 ID를 64자로 잘라 전달. long이면 <code>prompt_cache_retention:</code> <code>24h</code>	<code>prompt_tokens_detail.s.cached_tokens</code> . DeepSeek의 <code>prompt_cache_hit_tokens</code> , Kimi의 최상위 <code>cached_tokens</code> 도 같은 필드로 정규화	쓰기 토큰은 보고되지 않는 경우가 많아 <code>cacheWrite</code> 가 0으로 남는다. 키는 라우팅 힌트이며 캐시 경계를 지정하지 않는다.

경로	요청에 넣는 것	사용량 해석	유의점
Google Generative AI / Vertex	없음. 서버의 암묵적 캐시에 의존	<code>cachedContentTokenCount</code> 를 <code>cacheRead</code> 로, 나머지를 <code>input</code> 으로 계산	TTL을 알 수 없어 TTL 기반 대기 예산이 적용되지 않는다.
Bedrock Converse	<code>system</code> 과 메시지에 <code>cachePoint</code> . 모델이 지원하면 1시간 TTL	Anthropic과 동일한 읽기·쓰기 구분	<code>supportsPromptCaching</code> 판정을 통과한 모델에 한정된다.
Mistral Conversations	<code>promptCacheKey</code> y 에 세션 ID	<code>cached_tokens</code> 계열 필드를 정규화	명시적 affinity가 있으면 키를 넣지 않는다.

표는 요청 조립 코드에서 확인한 기본 동작이다. 게이트웨이나 호환 provider는 `compat` 설정으로 개별 항목을 끄거나 켤 수 있으므로, 같은 API 형식이라도 모델 항목의 설정에 따라 실제 동작이 달라진다.

캐시를 유지하기 위한 하네스 장치

체크포인트가 캐시를 만든다면, 아래 장치들은 만들어진 캐시가 낭비되지 않도록 시간과 요청 형태를 관리한다. 앞 절의 축소 정책 가운데 히스테리시스와 결정적 변환도 같은 목적에 속한다. ^[16]

270s

TTL - SAFETY

BUFFER

TTL을 넘기지 않는 도구 대기

`promptCache.cacheAwareTimeouts` 가 기본으로 켜져 있어, 전경 도구가 응답을 기다리는 시간을 모델의 캐시 TTL에서 30초를 뺀 값으로 제한한다. 5분 TTL이면 270초이다. 한도에 도달한 `bash` 명령은 죽이지 않고 백그라운드 세션으로 넘겨 다음 모델 호출이 캐시

3 / \$0.05

KEEPALIVE

DEFAULT OFF

유휴 중 캐시 갱신

`cache-keepalive` 확장은 세션이 유휴 상태일 때 TTL 만료 전에 warm ping을 보내 캐시를 갱신한다. 기본은 꺼져 있다. 커더라도 세션당 3회와 0.05달러 상한을 지킨다. Ping 간격은

prefix

CACHE-FRIENDLY SUMMARY

요약 요청의 prefix 재사용

실험 기능이 켜진 경우, 요약 생성 요청은 대화를 텍스트로 직렬화하는 대신 활성 provider 컨텍스트와 정확히 일치하는 prefix를 그대로 보낸다. 요약할 구간이 현재 요청의 앞부분과 `isDeepStrictEqual` 로 일치할 때에만 적용된다. 이 경우 요약 요청 자체에도 캐시 단가가

만료 전에 나가게 한다. TTL을 알 수 없는 모델에는 예산이 적용되지 않는다.

TTL에서 30초를 뺀 안전 대기 시간에 다시 60초 여유를 둔 값이다. 직접 Anthropic 연결에서만 동작하고 대기 메시지가 있거나 에이전트가 바쁘면 중단한다. 이 확장을 켜지 않았다면 100%에 가까운 표시와는 관련이 없다.

적용될 여지가 있다. 일치하지 않으면 기존 직렬화 방식으로 되돌아간다.

캐시가 깨지는 지점과 감지

`cache-stats.ts` 는 assistant 메시지마다 직전 요청과 비교해 miss를 계산한다. 직전 prompt 토큰과 현재 prompt 토큰 중 작은 값에서 `cacheRead` 를 뺀 값이 1024토큰을 넘으면 miss로 집계한다. 이때 직전 요청 이후 경과 시간과 모델 변경 여부를 함께 기록한다. 경과 시간이 5분을 넘으면 TTL 만료를 원인 후보로 표시한다. 캐시 활동을 한 번도 보고하지 않은 provider에서는 miss를 세지 않는다.

```
missedTokens = min(prevPromptTokens, promptTokens) - cacheRead
counted if missedTokens > 1024
```

구현에서 확인되는 prefix 변경 원인은 다음과 같다. 압축 적용은 활성 문맥을 통째로 바꾸므로 그 직후 호출은 새 내용으로 취급되어 miss 집계에서 제외된다. 모델 전환은 전체 prompt를 다시 과금하므로 miss로 센다. 앞 절의 축소 정책에서는 50% gate를 처음 넘는 순간, clearable 도구 결과의 최근 6개 창이 밀리는 순간, emergency latch가 켜지는 순간에 prefix 중간이 바뀐다. system prompt에는 `Current date` 가 포함되므로 날짜가 바뀐 뒤 prompt를 다시 조립하면 system 블록의 캐시도 새로 쓰인다.

정책과 적중률의 관계. 높은 적중률은 체크포인트 배치와 큰 고정 prefix가 만든다. 컨텍스트 정책은 그 prefix를 불필요하게 바꾸지 않도록 설계되어 있다. 정책이 적중률을 생성한다고 서술하면 인과가 뒤바뀐다. 정책의 효과를 보려면 세션 평균이 아니라 압축 직후와 축소 단계 전환 직후 호출의 footer 값과 cache waste를 봐야 한다.

앞선 정책을 모든 모델 호출에 공통으로 적용되는 절차로 읽어서는 안 된다. 이 하네스가 요청 메시지를 직접 구성하는 경로와 SDK가 자체 세션을 유지하는 경로에서는 컨텍스트 관리의 주체가 다르다. 코드의 소유권 판정은 어느 쪽이 자동 압축을 담당하고 어떤 요청 변환을 생략할지 결정한다. 아래 표는 이 차이에 따른 주요 기본 경로를 비교한다. ^[11]

경로	요청 축소	선행·유휴 요약	주의할 경계
이 하네스 일반 관리	기본 순서는 admission → reduction → emergency이다.	Pre-turn speculative와 유휴 warm 생성·적용 경로를 사용할 수 있다.	각 경로의 실행은 설정, 사용량, job의 상태와 breaker 조건을 받는다.
OpenAI remote 대상	일반 reduction을 기본적으로 제외한다. 이 하네스 관리 경로의 admission과 emergency는 유지한다.	Pre-turn speculative와 임계값 미만의 idle warm은 제외한다. 임계값 이상의 local idle 경로는 유지한다.	Blocking/core 경로는 remote compaction을 먼저 시도한다. 대상 판정에 해당하는 모델에 한정된다.
resident Claude SDK	SDK가 컨텍스트를 소유하면 이 하네스의 축소 파이프라인은 적용하지 않는다.	이 하네스의 자동 압축과 degradation 감시도 물러난다.	<code>resumeMode: off</code> 또는 compaction 모델 override가 소유권을 바꿀 수 있다. 수동 압축은 별도 경로이다.

표는 모든 분기를 열거한 상태 전이표가 아니라 주요 경로의 기본 구분이다. Breaker fallback 같은 조건은 일반 reduction gate를 별도로 우회할 수 있다. 각 enabled 플래그의 검사 위치가 같지 않으므로, 하나의 설정으로 모든 선행·유휴 경로가 대칭적으로 제어된다고 가정해서는 안 된다.

특히 OpenAI remote 대상이라는 표현을 모든 OpenAI 모델과 동일시하면 적용 범위를 잘못 판단하게 된다. 마찬가지로 resident Claude SDK의 기본 동작만 보고 이 하네스가 해당 provider에서 어떤 압축도 수행하지 않는다고 결론 내릴 수는 없다. 소유권을 바꾸는 설정과 수동 실행 경로가 있기 때문이다. 실제 세션의 정책은 모델 이름이 아니라 연결 방식과 compaction 설정의 조합으로 결정된다.

Codex와 Claude Code의 구현과 비교

컨텍스트 관리와 프롬프트 캐시는 이 하네스만의 문제가 아니다. 같은 과제를 풀고 있는 두 코딩 에이전트 구현체와 나란히 놓으면 어떤 선택이 공통 설계이고 어떤 것이 이 구현의 특징인지 드러난다. Codex는 오픈소스라 구현 자체를 대조할 수 있고(clone HEAD `d4205609`), Claude Code는 소스가 비공개라 공식 문서와 로컬 세션의 usage 필드 관측으로 한정한다. Claude Code의 행은 문서가 보장하는 범위의 서술이며 코드 수준의 단정이 아니다. ^[17] ^[18]

세 구현체 모두 공통의 토대 위에 있다. 요약에는 현재 대화의 모델을 그대로 쓴다. 요약 입력에는 공통 요약 지시를 마지막 사용자 메시지로 붙인다. 최근 대화 구간은 일부 보존한다. 캐시에는 세션 식별자나 prefix 일치가 관여한다. 차이는 토대 위의 분기에서 생긴다. Codex에는 세 경로가 있다. 원격 compaction을 지원하는 모델이면 서버에 CompactionTrigger 아이템을 보내 서버가 만든 Compaction 출력을 받는다. Feature::TokenBudget이면 요약 없이 새 컨텍스트 윈도우를 설치한다. 둘 다 아니면 로컬 요약을 수행한다. Claude Code는 로컬 요약 경로 하나에 서버 측 context editing이 병존한다. 이 하네스는 요약 이전에 결정적 축소 층을 둔다. ^[17] ^[18]

비교축	이 하네스	Codex	Claude Code
축소 트리거	적응 임계값(창 크기 ·직전 절약률) + 미완성 job 유예 + 하드 리밋의 다층 판정	한도 = 창의 90%, 하드캡 95%. ContextWindowExceed ed 에러 재시도, 턴 후 임계값(기본 비활성), 수동 /compact, 모델 전환	창 한계 도달 시 자동(~1M 모델에서 ~967K). /autocompact 또는 CLAUDE_CODE_AUTO_C OMPACT_WINDOW로 조정, 수동 /compact
요약 생성	현재 세션과 겹쳐 미리 생성 (speculative·idle) → 검증 후 적용 또는 보관	로컬·원격·토큰예산 3분기. 로컬은 동기 스트리밍으로 즉시 새 이력 설치	동기 요약 요청(공통 프롬프트에 요약 지시를 마지막 user로 부가) 후 즉시 설치
요약 이후 문맥	요약 + 보존 메시지 + restoration	로컬: 최근 user 메시지 최대 20k토큰 + 요약. 원격	요약 + CLAUDE.md·메모리· 계획·git 스냅샷 재주입 +

비교축	이 하네스	Codex	Claude Code
	라벨(파일 경로·작업·스킬 이름)	v2: 최근 64k토큰 보존 + Compaction 출력	최근 수정 파일 5개 재읽기 + 스킬 본문(5K/25K 상한)
도구 출력 처리	요청마다 결정적으로 줄임(5% 예산·접기·마커). 세션 원본은 유지	기록 시점에 모델별 truncation_policy로 중간 생략(exec 기본 10k토큰). 생략은 라이브 전용, rollout 원본 보존. 제거는 컴팩션 시	서버 측 context editing(베타): 100K에서 오래된 tool uses를 placeholder로 치환. 클라이언트 이력은 유지, prefix 캐시 무효화
캐시 고정 수단	Anthropic cache_control + 그 외 provider의 세션 키; 변형을 최소화하는 래치·결정적 변환	모든 요청에 prompt_cache_key(세션 ID). WebSocket은 previous_response_id 증분 + x-codex-turn-state 스티키 라우팅	tools→system→messages prefix 일치 + cache_control 체크포인트. 구독이면 메인 1h·헬퍼 5m, API 키면 5m TTL 버킷
대기 숨기기	임계값 앞 lead에서 생성 시작, 유틸 warm 생성·적용	없음. 임계값 도달 시 즉시 요약	없음(문서·관측 기준). 임계값 도달 시 즉시 요약
사용자 표시	footer CH%, /session 누적값·cache waste	{percent}% context left, status-line context-used, "Compacting context"."Context compacted"	/context 명령, 컨텍스트 표시(문서 기준)

Codex 열은 클론 d4205609의 codex-rs 소스, Claude Code 열은 공식 문서와 usage 필드 관측에 근거한다. Codex는 원격 compaction 지원 모델에서 로컬 요약을 거치지 않고 Claude Code의 context editing은 서버가 처리하므로 클라이언트 측 변환 코드와 무관하다.

로컬 관측: usage 필드가 말해 주는 것

두 구현체의 세션 로그를 이 장비에서 읽었다. Claude Code 세션에서는 호출당

cache_read_input_tokens 가 최대 약 738,000까지 쌓이고

cache_creation_input_tokens 는 수천~만 토큰에 그쳐 한 세션 내에서는 거의 전량이 캐시 읽기로

처리됐다. usage에 `ephemeral_1h` · `ephemeral_5m` 필드가 구분돼 나타나 TTL 버킷이 실제로 다르게 기록됨을 보인다. Codex의 rollout 로그는

`cached_input_tokens` · `cache_write_input_tokens` 를 누적 기록한다.

관측의 해석에는 한계가 있다. 표본 세션에서는 두 구현체 모두 자동 요약 이벤트를 찾지 못했고 Claude Code의 관찰 대상 세션은 `context_management: null` 로 context editing이 비활성이었다. 이는 기능의 부재가 아니라 해당 세션이 기능 경계에 도달하지 않았거나 설정이 꺼져 있었음을 의미한다. 요약·편집의 실제 행동은 해당 세션이 임계값을 넘겼을 때만 드러나므로, 이 관측은 필드 구조와 캐시 사용 패턴의 확인에 한정한다. [19]

이 하네스의 차이는 어디에 있는가

대조한 두 구현체에서 이 하네스의 특징이 세 갈래로 드러난다. 첫째, **요약 이전의 결정적 축소 총**이다. Codex는 도구 출력을 기록 시점에 한 번 자른다. 지우는 일은 요약이 맡는다. Claude Code는 오래된 tool uses 치환을 서버에 맡긴다. 이 하네스는 provider 요청마다 규칙으로 도구 출력과 답변을 다시 줄인다. 그래서 같은 대화라도 요약 도달 시점이 늦어진다. 둘째, **요약 대기의 숨기**기다. Codex와 Claude Code의 자동 요약은 도달한 턴에 즉시 동기 실행되므로 그 호출이 늦어지는데, 이 하네스는 임계값 앞 lead와 유희 시간에서 미리 생성해 둔다. 셋째, **실패 관리**다. 요약 실패의 circuit breaker와 무텍스트 응답 감시에 해당하는 장치는 두 구현체에서 확인되지 않았다.

반대 방향의 차이도 있다. Codex는 원격 compaction을 지원하는 모델에서 요약 자체를 서버에 위임하고 Feature::TokenBudget으로 요약 자체를 생략하는 분기가 있어 더 큰 구조적 선택지를 둔다. Claude Code는 요약 이후 CLAUDE.md·메모리·최근 수정 파일 5개·스킬 본문을 디스크에서 다시 읽어 붙이는 복원 단계가 문서에 명시되어 있고 서버 측 context editing은 클라이언트 구현 없이 같은 효과를 낸다. 이 하네스의 restoration tracker가 라벨만 재주입해 세부 본문 복원은 하지 않는 대조점이다.

비교의 비대칭. Codex는 소스 전체를 읽을 수 있으므로 각 셀이 코드 확인 값이고 Claude Code는 문서와 usage 관측이 근거다. "문서에 없음"은 "구현에 없음"이 아니며 Claude Code 행의 부재 표시는 모두 관측 범위 한정으로 읽어야 한다. [20]

기존 설명은 요청 축소와 선행 요약 결합한 다층 구조를 대체로 올바르게 파악했다. 다만 일부 수치의 기준을 바꾸어 읽거나 특정 경로의 조건을 전체 정책의 보장으로 확대했다. 아래 항목은 결론에 영향을 주는 차이를 정리한 목록이다. '정정'은 구현과 다른 내용이며 '조건부'는 적용 범위나 추가 조건을 명시해야 성립하는 설명이다.

정정 "파일.스킬 본문을 복원해 재독을 줄인다"

Restoration tracker가 재주입하는 것은 **파일 경로·작업 종류·스킬 이름**이다. 본문을 다시 읽거나 스킬 지침을 로드하는 기능은 포함하지 않는다. 항목 선택도 최근순이 아니라 우선순위, 짧은 항목, 이름순으로 이루어진다.

관련 대상을 상기시키는 효과는 기대할 수 있지만 그 단서만으로 후속 작업에 필요한 본문을 확보했다고 볼 수는 없다. 파일 재독 횟수나 비용이 줄었다는 결론은 별도의 실행 관찰이 필요하다. 근거 [08]

정정 "800토큰 넘는 옛 답변을 원문의 30%로 축소한다"

남길 본문의 목표는 **설정 기준 800토큰의 30%, 약 240토큰**이다. 원문이 2000토큰이어도 600토큰을 남기는 계산이 아니며 최종 결과에는 축소 마커가 추가된다.

적용 범위도 최근 보호 구간 밖의 텍스트 전용 assistant 메시지로 제한된다. 축소 비율만 전달하고 이 조건을 생략하면 최근 답변이나 도구 호출이 포함된 메시지에도 같은 처리가 적용되는 것으로 오해할 수 있다. 근거 [04]

정정 "비효율 요약을 circuit breaker가 걸러낸다"

Ineffective 카운터와 실패 breaker는 현재 직접 연결되어 있지 않다. 수용된 요약은 먼저 성공으로 기록되어 실패 횟수를 초기화하며 비효율은 별도 카운트와 로그로 남는다. Cap 판정도 항상 허용하는 값을 반환한다.

따라서 주석에 나타난 차단 의도를 현재 실행 코드의 보장으로 인용할 수 없다. 실패를 반복한 작업의 cooldown과 성공했지만 절약량이 작은 요약의 집계는 서로 다른 장치이다. 근거 [09]

조건부 "50%에서 idle 요약을 만들고 바로 적용한다"

창의 50%는 일반적인 유틸 warm 생성의 하한이다. 완료 후 **idle apply에는 adaptive threshold 이상이라는 별도 조건**이 있고 세션이 계속 유틸 상태인지도 확인한다. 기본 200k 예시에서는 100k에서 생성했더라도 140k 미만이면 결과를 보관할 수 있다.

사용자가 생성 완료 전에 새 요청을 보내면 유틸 적용의 전제가 사라지는 경우도 있다. Blocking 경로가 미완성 job을 기다리는 경우도 있으므로, 이 정책에서 무지연을 보장한다고 결론 낼 수는 없다. 근거 [07]

조건부 “max(keepRecent, 8192)만큼 자라면 결과를 폐기한다” —

시작 이후 토큰 증가량이 해당 값을 초과하는지 검사하는 조건은 유틸 warm job의 교체 판단에 쓰인다. 값에 도달한 경우와 초과한 경우를 구분해야 하며 시간 경과에 의한 만료 조건도 아니다.

Blocking 소비 등 다른 경로는 generation, revision, anchor의 유효성을 확인한다. 토큰 증가량 공식을 모든 적용 경로에 공통인 폐기 제한으로 설명하면 실제로 유효한 결과의 처리까지 잘못 예측하게 된다. 근거 [07]

조건부 “히스토리 손실 없이, 항상 더 싸고 빠르다” —

저장 원본의 보존과 요청 정보의 보존은 다르다. 이번 모델 요청에서 생략한 본문은 모델 입력에 포함되지 않는다. 발체와 마커가 필요한 정보를 충분히 유지했는지는 작업에 따라 달라질 수 있다.

실제 비용을 평가하려면 요청량 감소뿐 아니라 요약 생성과 폐기, provider 캐시 적중, 생략한 정보의 재탐색도 함께 관찰해야 한다. 본 보고서는 실행 조건을 분석한 자료이며 비용이나 지연의 실측 결과를 포함하지 않는다.

조건부 “이 하네스는 compaction을 더 늦게 한다” —

직전 yield 조정이 없는 기본 200k 창에서 이 하네스의 요약 admission 임계값은 **140,000토큰**이다. 비교한 upstream pi의 기본 조건은 **183,616토큰** 초과이므로, 이 하네스는 이 기준에서 요약을 더 일찍 고려한다.

다만 이 비교는 admission 기준의 비교이다. 이 하네스의 생성 시작점은 그보다 앞설 수 있고 미완성 job에 대한 grace는 blocking을 늦출 수 있다. 생성, 적용, blocking의 시점을 분리해야 두 시스템을 같은 기준으로 비교할 수 있다. 근거 [12]

조건부 “세션 내내 캐시 적중률이 100%에 가까운 것은 컨텍스트 정책 덕분이다” —

Footer의 수치는 가장 최근 provider 호출 하나의 `cacheRead / (input + cacheRead + cacheWrite)` 이다. system prompt와 tool 정의라는 큰 고정 prefix가 매 호출 다시 읽히므로, 두 번째 호출부터 90%대, 이후 99%에 가깝게 수렴한다. 이는 prefix cache의 산술적 성질이며 세 번의 사용자 요청은 그보다 훨씬 많은 호출을 포함한다.

이 수렴을 가능하게 하는 것은 Anthropic 경로의 `cache_control` 체크포인트와 다른 provider의 세션 키이다. 컨텍스트 정책은 히스테리시스와 결정적 변환으로 그 prefix를 불필요하게 바꾸지 않도록 하는 쪽이다.

11 REAL SESSION DATA

내 세션 한눈에

앞 장들은 정책의 구조를 설명했다. 이 장은 같은 구조가 실제로 돌아간 기록을 펼쳐 보인다. 이 보고서를 만든 컴퓨터에 쌓인 개인 세션 기록 111개를 집계해, 그중 메시지가 가장 많은 세션을 골랐다. 4,883개 메시지는 컨텍스트 창 하나에 한 번에 담기 어려운 양으로, 앞 장에서 설명한 축소와 요약이 실제로 쓰이는 규모를 보여 준다.

메시지가 가장 많은 세션

4,883 개 메시지

2026-09-27 22:20 ~ 2026-09-30 00:29 (KST) · 약 50시간 동안 · 기록 크기 약 18.8MB

사용자 메시지

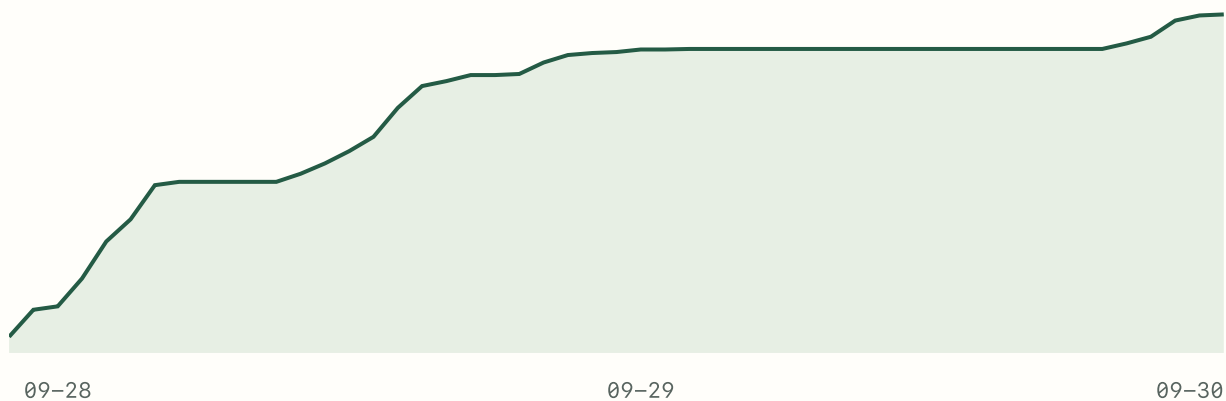
61

어시스턴트 답변

2,387

도구 결과

2,435



세로축은 누적 메시지 수, 가로축은 세션 시작부터의 시간이다. 곡선이 가파르게 오르는 구간에서는 짧은 시간에 메시지가 많이 쌓였다.

1위 09-27 → 09-30		4,883
2위 09-25 → 09-27		2,247
3위 08-30 → 08-31		2,030
4위 09-29 → 10-01		1,103
5위 09-29		1,048

값은 로컬 세션 기록 파일에서 직접 집계했다. 메시지 수는 기록에 남은 메시지 항목 수이며, 사용자·어시스턴트·도구 결과를 모두 포함한다. 세션 이름과 식별자는 표시하지 않는다.

분석 근거와 결론의 범위

보내는 토큰을 줄이고 보낸 토큰을 캐시 단가로 받으며
요약 대기를 작업 시간과 겹친다.
세 축의 실제 절감 효과는 별도 측정이 필요하다.

효율적인 에이전트 하네스라는 관점에서 이 구현은 세 가지 변수를 각각 다른 장치로 다룬다. 요청량은 admission, reduction, emergency prune과 compaction이 줄인다. 과금 단가는 provider별 캐시 표시와 세션 키, TTL을 넘기지 않는 대기 예산, 요청 형태를 안정시키는 히스테리시스가 관리한다. 대기 시간은 speculative 생성, grace band, 유티 warm-up이 줄이려 한다. 세 축은 서로 영향을 준다. 예를 들어 압축은 요청량을 줄이지만 그 직후 호출의 캐시를 비운다. 축소 단계의 전환도 요청량을 줄이면서 prefix를 바꾼다. 어느 한 축의 지표만으로는 하네스의 효율을 판단할 수 없다.

분석 기준 커밋은 `f81aee565f2aeb6d774c93801596e523c017c4c5` 이다. 정책 디렉터리에는 TypeScript 파일 47개, 8537줄이 있으며 해당 경로의 2026-04-27 이후 이력은 216커밋으로 확인했다. 이러한 수치는 정책 구현의 범위와 변경 이력을 보여 주지만 압축 품질이나 실행 효율을 입증하는 지표로 사용할 수는 없다.

분석에서는 정책 함수의 조건과 반환값을 읽고 실제 이벤트 후와 호출부에서 그 값이 어떻게 소비되는지 대조했다. 관련 테스트는 기대 동작을 확인하는 보조 근거로 읽었다. 특히 함수나 주석의 이름이 암시하는

기능을 그대로 채택하지 않고 실행 경로가 그 기능을 실제로 강제하는지 확인했다. 비효율 집계와 breaker의 비연결, 유틸 생성과 적용 조건의 차이는 이 대조에서 드러난 사례이다.

원본 코드의 테스트 실행, 실제 모델 요약 호출, cache hit·지연·비용 벤치마크는 수행하지 않았다. 보고서의 대화형 시각화는 **명시된 가정 아래 공식과 상태 판정을 재현하는 설명 도구**이다. 수식을 재현했다는 사실만으로 특정 작업에서 요약 품질이 유지되거나 총비용이 감소한다고 결론 내릴 수는 없다.

이 구현은 요청에 포함되는 정보를 규칙적으로 줄이는 방향과, 요약이 필요해지기 전에 생성을 시작해 대기의 일부를 다른 작업 시간과 겹치는 방향으로 설계되어 있다. 그 과정에서 원본 기록은 유지되지만 현재 요청의 정보는 생략된다. 선행 생성한 결과를 사용하지 못할 가능성도 남는다. 실제 효과를 평가하는 후속 검증에서는 줄어든 요청량과 함께 요약 요청 수, 폐기된 결과, 재탐색, 사용자 대기 시간을 같은 작업 조건에서 관찰해야 한다. 이는 향후 측정의 기준이며 본 보고서가 확보한 실측 결과는 아니다.

근거의 재현 범위. 아래 원문 링크는 분석에 사용한 커밋에 고정되어 있다. 보고서는 외부 자산을 내려받지 않고 열 수 있으나 GitHub 원문을 조회하려면 인터넷 연결이 필요하다. 정책 파일의 공통 경로는 `packages/coding-agent/src/core/extensions/builtin/compaction/` 이다.

01 원본 복제와 provider 요청 변환

`emitContext`가 원본 메시지를 복제한 뒤 혹은 실행하고 `buildProviderContext`가 변환 결과를 provider 요청에 사용한다. 세션 원본과 요청 투영을 구분하는 근거이다.

```
runner.ts:1560 agent-loop.ts:403
```

02 실행 후과 파이프라인

pre-turn admission, context projection, turn-end recovery, agent-end warming.

```
index.ts:825 context-pipeline.ts:29
```

03 도구 출력 admission

텍스트 예산, 60/20 발체, 결과 내 여러 텍스트 블록의 예산 배분.

```
tool-admission.ts:10 orchestration.ts:97
```

04 일반 context reduction

50% gate, builtin 옵션, 240토큰 본문 목표, 텍스트 전용 조건, clearable 목록.

```
context-reduction.ts:130 context-reduction.ts:408 context-reduction.test.ts:77
```

05

긴급 축소와 hysteresis

95% 초과 진입·85% 이하 해제, 4096바이트와 800/400자, 출력 reserve 계산.

emergency-prune.ts:61 tool-truncation.ts:4 extension-wiring.ts:18

hysteresis.test.ts:79

06

임계값·reserve·lead·reminder

창별 baseline, 직전 yield 조정, reserve scaling, grace 경계와 generation별 안내.

policy.ts:61 speculation-lead.ts:1 token-budget-reminder.ts:39

compaction-settings.ts:20

07

선행 생성·유휴 적용·anchor

생성과 적용 조건의 분리, idle 교체 판단, append를 허용하는 경계 검증.

idle.ts:22 index.ts:240 orchestration.ts:82 speculative.ts:428

warm-anchor.ts:17

08

복원 payload의 실제 내용

본문 아닌 경로·operation·skill 이름, 선택 순서와 합산 예산.

restoration-tracker.ts:175 restoration-tracker.ts:204

09

실패 breaker와 ineffective telemetry

실패 3회·60초, success reset, ineffective 기록, 항상 허용하는 cap.

circuit-breaker.ts:4 yield.ts:69 index.ts:768 per-turn-cap.ts:23

10

무텍스트 감시와 복구 시도

turn-end 감시 창, 비-assistant reset, 사이클당 gate와 누적 시도 ceiling.

degradation-monitor.ts:72 index.ts:986

11

Provider 소유권과 예외

resident SDK stand-down, remote 대상 모델 판정과 해당 경로의 예외.

lane-policy.ts:99 openai-remote-model.ts:62 index.ts:888 index.ts:1024

12

Upstream 비교 기준

저장소가 고정된 기준은 pi v0.84.4 · f41f8046이다. 해당 compaction.ts는 1012줄이며 기본 요약 조건은 창에서 reserve 16,384를 뺀 값을 초과할 때 성립한다. 기존 설명의 '865줄'은 이 비교 기준과 일치하지 않는다.

`.github/upstream.json` pi compaction.ts:235

13 캐시 적중률의 정의와 표시

세션 관리자가 assistant 메시지마다 최근 비율을 덮어쓰고 footer는 10% 이상일 때 표시한다. 세션 정보 화면은 누적 합계와 cache waste를 별도로 계산한다.

`session-manager.ts:1172` `footer.ts:185` `interactive-mode.ts:8890`

`cache-stats.ts:56`

14 Anthropic 체크포인트와 TTL

system·tools·마지막 user 블록의 cache_control, tool loop 시 직전 체크포인트 보존, effort 마커 이후 재표시, short/long retention 판정.

`anthropic-messages.ts:99` `anthropic-messages.ts:951` `anthropic-messages.ts:2015`

`anthropic-messages.ts:2464` `prompt-cache-ttl.ts:11`

15 다른 provider의 캐시 키와 사용량 정규화

OpenAI 계열의 prompt_cache_key와 24시간 retention, cached_tokens 계열 필드의 통합, Google의 cachedContentTokenCount, Bedrock cachePoint, Mistral promptCacheKey.

`openai-completions.ts:1063` `openai-completions.ts:1850` `openai-prompt-cache.ts:1`

`google-generative-ai.ts:264` `bedrock-converse-stream.ts:881`

`mistral-conversations.ts:529`

16 캐시 유지 장치

TTL 기반 도구 대기 예산, 기본 꺼짐인 keepalive의 회수·비용 상한, 실험 기능의 cache-friendly 요약 prefix 재사용, system prompt의 날짜 항목.

`prompt-cache-budget.ts:37` `settings-manager.ts:797` `cache-keepalive/index.ts:101`

`agent-session.ts:5455` `compaction-execution.ts:218` `dynamic-prompt/build.ts:63`

17 Codex compaction 분기와 한도

run_auto_compact의 3분기(토큰예산·원격·로컬), 한도 = 창의 90%·하드캡 95%, 요약 대상과 보존 범위, post-turn 임계값 기본 0.

`turn.rs:1488` `compact.rs:78` `openai_models.rs:525`

18

Claude Code auto-compact-context editing-캐시

/autocompact와 CLAUDE_CODE_AUTO_COMPACT_WINDOW, /compact의 동기 요약 구조와 재주입 항목, 서버 측 context editing(clear_tool_uses-clear_thinking), TTL 버킷.

```
claude-code docs: context docs: context editing docs: prompt caching
```

19

로컬 usage 관측

Claude Code 세션의 cache_read_input_tokens-cache_creation_input_tokens-ephemeral_* 필드와 context_management:null, Codex rollout의

cached_input_tokens-cache_write_input_tokens-token_count 이벤트.

로컬 관측: ~/.claude/projects/-Users-gahn-Works-01-*/01a031de*.jsonl,

~/.codex/sessions/2026/09/20/rollout-*.jsonl, ~/.codex/config.toml

20

Codex 캐시 고정 수단

모든 요청의 prompt_cache_key(세션 ID), previous_response_id 증분 델타, x-codex-turn-state 스티키 라우팅, store:false와 encrypted_content.

```
client.rs:356 model-provider-info.rs:380 tool_call.rs:138
```

CONTEXT ATLAS

[PDF 내려받기](#)[처음으로 ↑](#)

분석 기준 2026.09.10 · 단일 파일 / 외부 라이브러리 없음